



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización y Arquitectura de Computadoras

Reporte - Construcción de Máquinas de estados Usando Memorias con Direccionamiento Implícito

INTEGRANTES EQUIPO 8:

- Saldaña Navarrete Andrea - *Grupo Teoria:* 2
- Tepal Briseño Hansel Yael - *Grupo Teoria:* 1
- Ugartechea González Luis Antonio - *Grupo Teoria:* 1

NOMBRE DELPROFESOR: Ing. Diana Karen Pérez Pérez
GRUPO: 06
SEMESTRE: 2027-1

FECHA DE ENTREGA: 5 de noviembre del 2026

Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando direccionamiento de memorias con el método de direccionamiento implícito.

Introducción

En esta práctica se aborda la construcción de máquinas de estados utilizando memorias mediante el método de direccionamiento implícito. A diferencia de otras arquitecturas, este método se caracteriza por emplear un único campo de liga en la memoria de control. La lógica de transición funciona evaluando una variable de entrada (seleccionada mediante un campo de prueba), la cual se compara con un bit de control conocido como VF (Verdadero-Falso). Mediante compuertas lógicas XOR y XNOR, esta relación determina si el sistema toma una trayectoria de incremento sumando una unidad al estado presente, o si toma una trayectoria de carga para realizar un salto hacia la dirección almacenada en el campo de liga. Para esta implementación, se utilizó una variante arquitectónica que sustituye el contador tradicional por dos registros independientes: un registro µPC para almacenar el estado presente más uno, y un registro dedicado a la liga. Esta estructura es de gran importancia teórica y práctica, ya que constituye la base fundamental del secuenciador utilizado en la Unidad de Control de Procesos de una computadora.

Desarrollo

1. Diseñe una carta ASM con las siguientes características:

- Estados: 6 hasta 16
- Entradas: 3
- Salidas: 4
  - 2 con lógica estándar
  - 2 con lógica negada
- Colocar al menos 3 salidas condiciones si se puede.

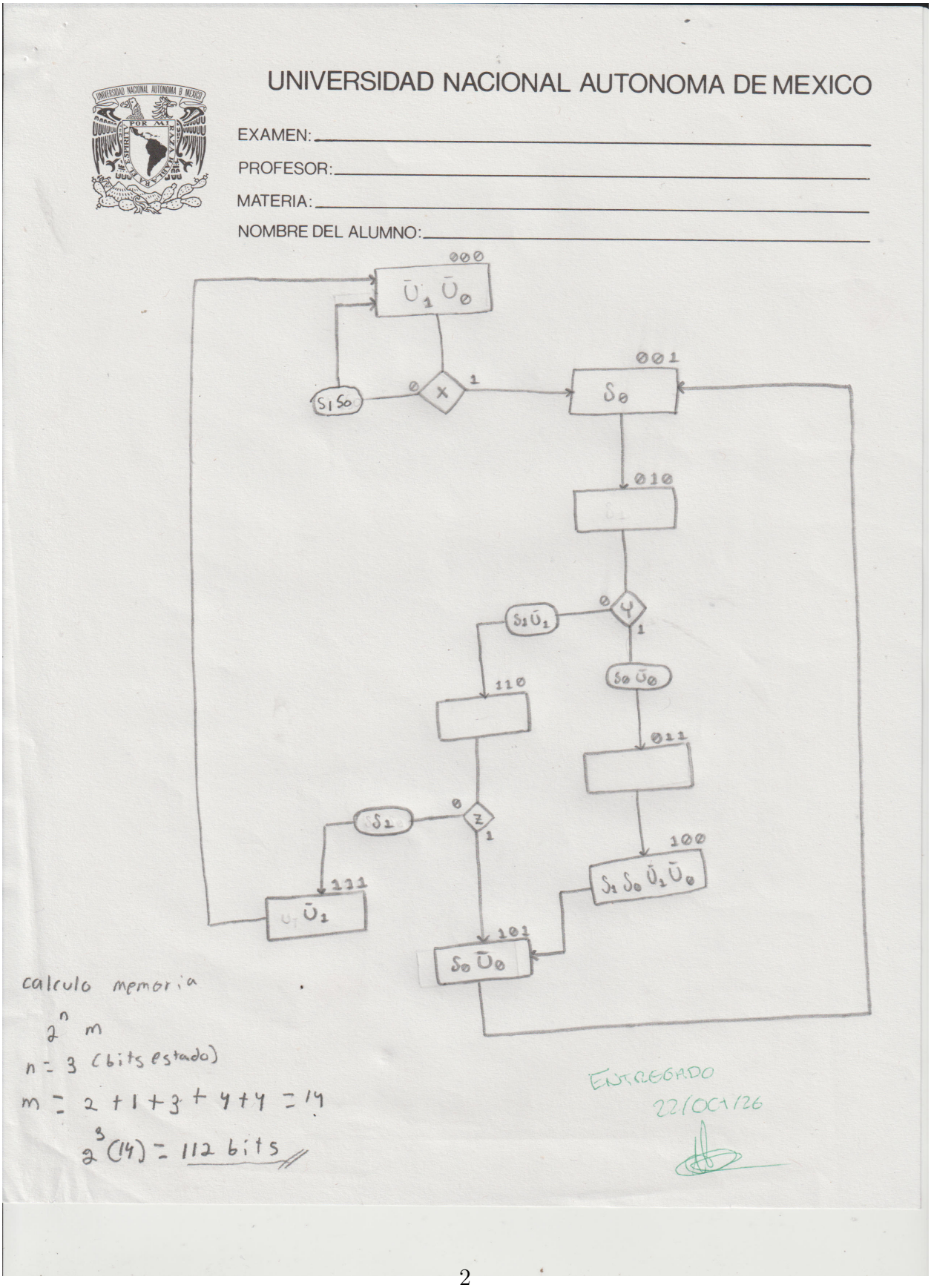


Figure 1: Carta ASM firmada por la profesora.

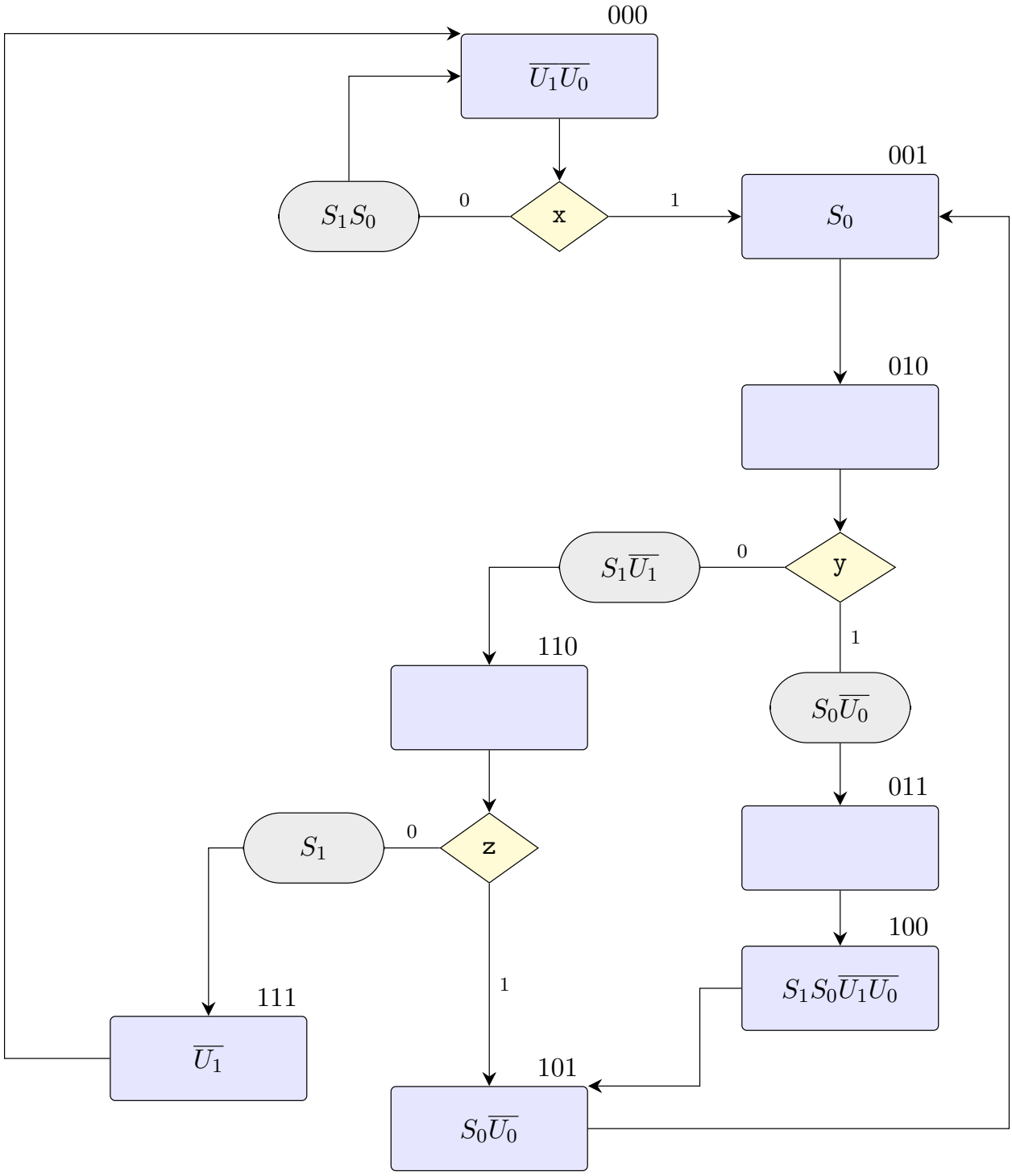


Figure 2: Carta ASM de 8 estados con direccionamiento Implícito.

2. Encuentre el tamaño de la memoria necesaria y el contenido de memoria utilizando el direccionamiento entrada-estado.

Encuentre el tamaño de la memoria necesaria y el contenido de memoria utilizando el direccionamiento por trayectoria. Recuerde que antes de construir la tabla se debe asignar a cada estado de la carta ASM una representación binaria. Así mismo, recuerde que para cada estado es necesario considerar todas las posibles combinaciones de las variables de entrada, aun cuando algunas de ellas no se utilicen en determinado estado.

Cálculo del tamaño de memoria

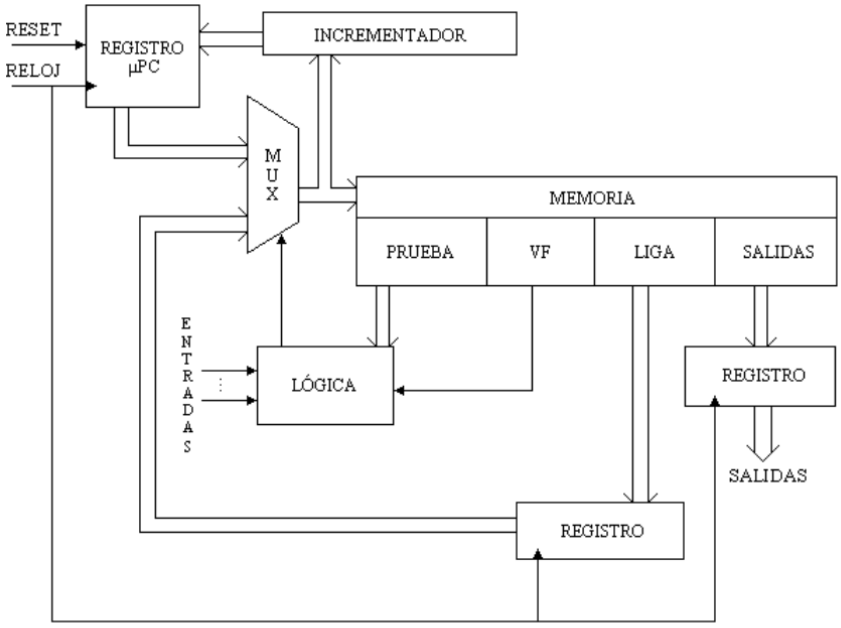


Figura 3.12. Variante del direccionamiento implícito.

Figure 3: Esquema del Direccionamiento Implícito.

Tamaño de memoria

$$2^n \cdot m$$

En donde:

- n = número de bits del estado presente.
- m = prueba + liga falsa + liga verdadera + salida falsa + salida verdadera.

En esta máquina se tienen 10 estados. Por lo tanto, se necesitan 4 bits para representarlos, ya que $2^4 = 16$ combinaciones posibles.

$$n = 4 \text{ bits}$$

La dirección de memoria tendrá 4 bits y podrá seleccionar 16 localidades. Para calcular la longitud de la palabra (m), se suman los bits de los campos requeridos por la arquitectura:

- **Prueba:** 2 bits (para seleccionar entre las entradas $w = 00, x = 01, y = 10, z = 11$).
- **Liga Falsa:** 4 bits (estado siguiente si la evaluación es 0).
- **Liga Verdadera:** 4 bits (estado siguiente si la evaluación es 1).
- **Salida Falsa:** 4 bits (estado de las salidas $S_1, S_0, \overline{U_1}, \overline{U_0}$ si la evaluación es 0).
- **Salida Verdadera:** 4 bits (estado de las salidas si la evaluación es 1).

$$m = 2 + 4 + 4 + 4 + 4 = 18 \text{ bits}$$

Por lo tanto, el tamaño de memoria necesario es:

$$2^4 \cdot 18 = 16 \cdot 18 = 288 \text{ bits}$$

Contenido de la memoria

Estado Presente			**Prueba**		**VF**	**Liga**			**Salidas Carga**				**Salidas Inc.**			
$P_2(t)$	$P_1(t)$	$P_0(t)$	K_1	K_0		L_2	L_1	L_0	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$
0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
0	0	1	*	*	*	0	1	0	0	1	1	1	0	1	1	1
0	1	0	0	1	0	1	1	0	1	0	0	1	0	1	1	0
0	1	1	*	*	*	1	0	0	0	0	1	1	0	0	1	1
1	0	0	*	*	*	1	0	1	1	1	0	0	1	1	0	0
1	0	1	*	*	*	0	0	1	0	1	1	0	0	1	1	0
1	1	0	1	0	1	1	0	1	0	0	1	1	1	0	1	1
1	1	1	*	*	*	0	0	0	0	0	0	1	0	0	0	1

Table 1: Tabla de transiciones y salidas de la máquina de estados.

3. Implementación en Quartus del direccionamiento implícito.

Una vez que haya obtenido el contenido de memoria, implemente el direccionamiento implícito utilizando el software de desarrollo Quartus y escriba el contenido de memoria obtenido.

Descripción de la implementación

La implementación se realizó en Quartus Prime utilizando el archivo `practica_5.bdf` como entidad de nivel superior. El diseño se organizó en bloques VHDL independientes: un divisor de frecuencia, un registro de estado, una memoria ROM, un divisor de datos, un multiplexor de entradas, un incrementador y dos multiplexores de selección: uno para la siguiente liga y otro para las salidas.

El registro de estado utiliza tres bits, por lo que puede representar los ocho estados de la máquina. Su salida se conecta a la entrada de dirección de la memoria ROM. Para cada estado, la memoria entrega una palabra de 14 bits con el formato

$$\underbrace{\text{Prueba}}_{2\text{ bits}}\ \underbrace{\text{VF}}_{1\text{ bit}}\ \underbrace{\text{Liga}}_{3\text{ bits}}\ \underbrace{\text{Salidas Inc.}}_{4\text{ bits}}\ \underbrace{\text{Salidas Carga}}_{4\text{ bits}}.$$

El bloque `divisor_datos` separa esta palabra en sus campos. El multiplexor `mux5` selecciona la entrada física que corresponde al campo `Prueba`; las entradas X, Y y Z corresponden a las variables externas y W se conecta a un nivel lógico alto para representar la condición constante. La señal obtenida controla los multiplexores de liga y de salida. Finalmente, el incrementador genera el estado siguiente secuencial, mientras que `mux1X2X1_3` selecciona entre el estado incrementado y la liga almacenada en memoria.

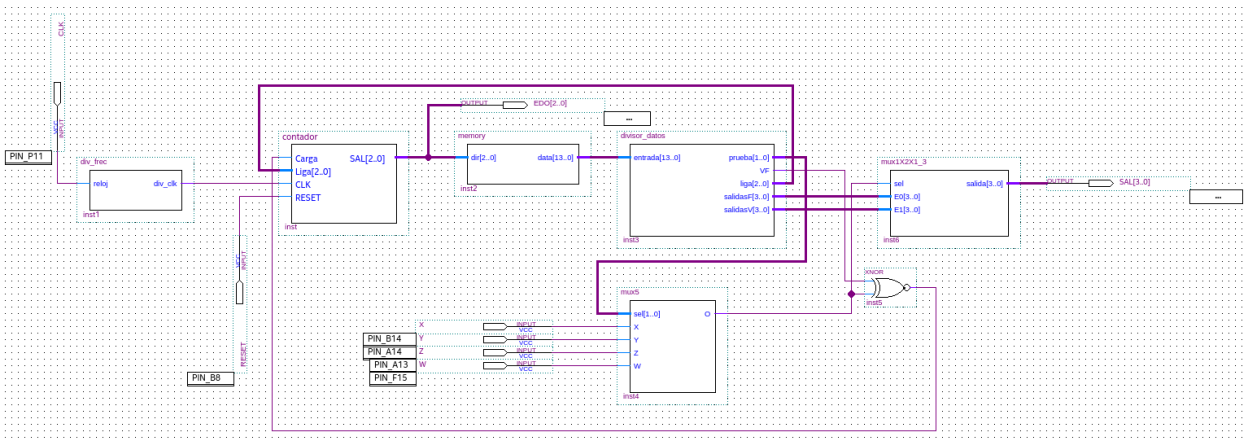


Figure 4: Esquemático de la implementación completa en Quartus.

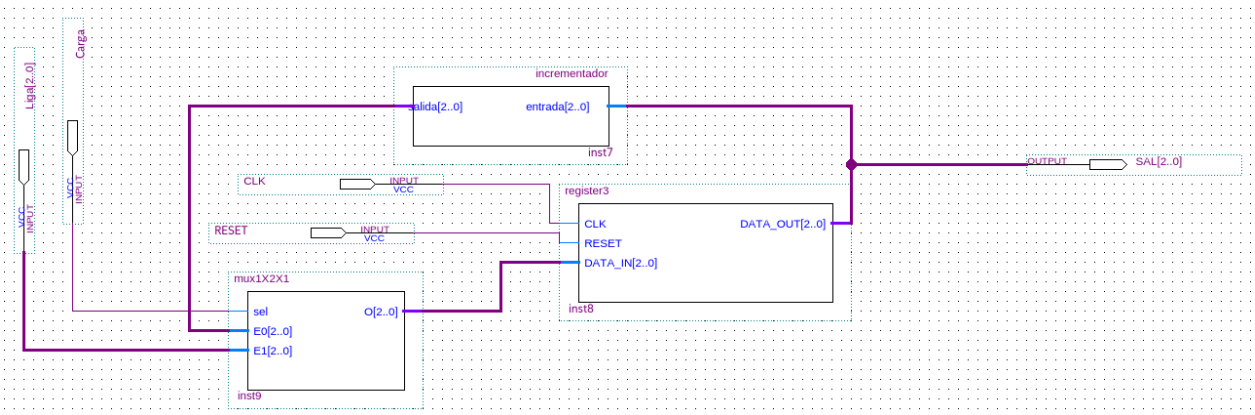


Figure 5: Esquemático del contador y registro utilizados para recorrer los estados.

Código de los módulos

El divisor de frecuencia reduce la frecuencia del reloj de la tarjeta para que los cambios de estado puedan observarse durante la prueba física.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity div_frec is
  Port (reloj : in std_logic;
        div_clk : out std_logic);
end div_frec;

architecture Behavioral of div_frec is
    signal pulso : std_logic := '0'; -- Señal para alternar el
       estado
begin
  process (reloj)
  variable cuenta : std_logic_vector (27 downto 0) := X"0000000";
  begin
    if rising_edge (reloj) then
      if cuenta = X"2FAF080" then
        cuenta := X"0000000";
        pulso <= not pulso; -- Cambia de 0 a 1 y viceversa cada 1
           segundo
      else
        cuenta := cuenta+1;
      end if;
    end if;
  end process;

```

```
  div_clk <= pulso; -- Asignas la señal en lugar del bit de cuenta
end Behavioral;
```

Listing 1: Divisor de frecuencia utilizado en Quartus.

El registro conserva el estado presente y lo actualiza en cada flanco de subida del reloj dividido. El reinicio activo en bajo devuelve la máquina al estado 000.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register3 is
  Port( CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        DATA_IN : in STD_LOGIC_VECTOR(2 downto 0); --Liga
        DATA_OUT : out STD_LOGIC_VECTOR(2 downto 0)--Edo Presente
        );
end register3;

architecture Behavioral of register3 is
  signal internal_value : std_logic_vector(2 downto 0) := B"000";
begin
  process (CLK, RESET, DATA_IN)
  begin
    if RESET = '0' then
      internal_value <= B"000";
    elsif rising_edge (CLK) then
      internal_value <= DATA_IN;
    end if;
  end process;

  process(internal_value)
  begin
    DATA_OUT <= internal_value;
  end process;
end Behavioral;
```

Listing 2: Registro de estado de tres bits.

La memoria ROM contiene las ocho palabras de control obtenidas a partir de la tabla de estados. La dirección corresponde directamente al estado presente.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity memory is
  Port( dir : in STD_LOGIC_VECTOR(2 downto 0);
```

```
      data : out STD_LOGIC_VECTOR(13 downto 0)
      );
end memory;

architecture Behavioral of memory is
  type mem is array(0 to 7) of std_logic_vector(13 downto 0);
  signal internal_mem : mem := (
    -- Orden físico: Prueba(2) & VF(1) & Liga(3) & Salidas Inc.(4) &
        Salidas Carga(4)
    "00" & '0' & "000" & "0000" & "1100", -- Estado 000: Liga
       corregida a 000
    "00" & '0' & "010" & "0111" & "0111", -- Estado 001
    "01" & '0' & "110" & "1001" & "0110", -- Estado 010
    "00" & '0' & "100" & "0011" & "0011", -- Estado 011
    "00" & '0' & "101" & "1100" & "1100", -- Estado 100
    "11" & '1' & "001" & "0110" & "0110", -- Estado 101: Salto
       incondicional forzado a 001 evaluando W(11)
    "10" & '1' & "101" & "1011" & "0011", -- Estado 110
    "00" & '0' & "000" & "0001" & "0001"  -- Estado 111
  );
begin
  process(dir)
  begin
    data <= internal_mem(conv_integer(unsigned(dir)));
  end process;
end Behavioral;
```

Listing 3: Memoria ROM con el contenido de la máquina.

El divisor de datos desempaqueta la palabra de 14 bits para alimentar los bloques de selección.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divisor_datos is
  Port( entrada : in STD_LOGIC_VECTOR(13 downto 0);
      prueba : out STD_LOGIC_VECTOR(1 downto 0);
      VF : out STD_LOGIC;
      liga : out STD_LOGIC_VECTOR(2 downto 0);
      salidasF : out STD_LOGIC_VECTOR(3 downto 0);
      salidasV : out STD_LOGIC_VECTOR(3 downto 0)
      );
end divisor_datos;

architecture Behavioral of divisor_datos is
begin
  process (entrada)
```

```
  begin
    prueba <= entrada(13 downto 12);
    VF <= entrada(11);
    liga <= entrada(10 downto 8);
    salidasF <= entrada(7 downto 4);
    salidasV <= entrada(3 downto 0);
  end process;
end Behavioral;
```

Listing 4: Separación de los campos de la palabra de memoria.

El multiplexor de entradas selecciona la condición que debe evaluarse según el campo `Prueba` almacenado en la ROM.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux5 is
    Port( sel : in STD_LOGIC_VECTOR(1 downto 0);
          X : in STD_LOGIC;
          Y : in STD_LOGIC;
          Z : in STD_LOGIC;
          W : in STD_LOGIC; -- VCC (1 Lógico)
          O : out STD_LOGIC
          );
end mux5;

architecture Behavioral of mux5 is
begin
    process (sel, X, Y, Z, W)
    begin
        case sel is
            when "00" => O <= X;
            when "01" => O <= Y;
            when "10" => O <= Z;
            when "11" => O <= W;
            when others => O <= '0';
        end case;
    end process;
end Behavioral;
```

Listing 5: Multiplexor de selección de entradas.

El incrementador produce el siguiente estado secuencial. El multiplexor de tres bits permite seleccionar entre este valor y la liga proporcionada por la memoria.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
```

```
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity incrementador is
  Port( entrada : in STD_LOGIC_VECTOR(2 downto 0);
        salida : out STD_LOGIC_VECTOR(2 downto 0)
        );
end incrementador;

architecture Behavioral of incrementador is
begin
  process (entrada)
  begin
    salida <= entrada + 1;
  end process;
end Behavioral;
```

Listing 6: Incrementador del estado.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux1X2X1 is
  Port( sel : in STD_LOGIC;
        E0 : in STD_LOGIC_VECTOR(2 downto 0);
        E1 : in STD_LOGIC_VECTOR(2 downto 0);
        O : out STD_LOGIC_VECTOR(2 downto 0)
        );
end mux1X2X1;

architecture Behavioral of mux1X2X1 is
begin
  process (sel, E0, E1)
  begin
    if sel = '0' then
      O <= E0;
    elsif sel = '1' then
      O <= E1;
    end if;
  end process;
end Behavioral;
```

Listing 7: Multiplexor para seleccionar la siguiente liga.

Por último, el multiplexor de cuatro bits selecciona el grupo de salidas correspondiente a la condición evaluada, entre las salidas de incremento y las salidas de carga.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```vhdl
entity mux1X2X1_3 is
  Port( sel : in STD_LOGIC;
       E0 : in STD_LOGIC_VECTOR(3 downto 0); -- Incremento
       E1 : in STD_LOGIC_VECTOR(3 downto 0); -- Carga (Liga)
       salida : out STD_LOGIC_VECTOR(3 downto 0)
       );
end mux1X2X1_3;

architecture Behavioral of mux1X2X1_3 is
begin
  process (sel, E0, E1)
  begin
    if sel = '0' then
      salida <= E0;
    else
      salida <= E1;
    end if;
  end process;
end Behavioral;
```

Listing 8: Multiplexor de las salidas de la máquina.

Discusión de Resultados

5. Simule su diseño para probar su funcionamiento. Recuerden que en sus simulaciones debe aparecer el contenido de la memoria además del estado presente.

Para todas las simulaciones, se modifico el archivo `divisor_frecuencia.vhd` a un valor muy pequeño para poder observar como cambia el estado presente y las salidas de la máquina. Se realizaron tres simulaciones con diferentes entradas para observar el comportamiento de la máquina.

De igual forma, en cada simulación se incluyo el primer ciclo cuando la entrada X es 0, para observar el comportamiento de la máquina cuando se encuentra en el estado inicial.

Simulación 1

En esta simulación se consideraron las entradas `X=1, Y=0, Z = 0 y W = 1`.

sec-1: Estado: `000` → Salida: `0000` → Estado siguiente: `000` →

sec-2: Estado: `000` → Salida: `1100` → Estado siguiente: `001` →

sec-3: Estado: `001` → Salida: `0111` → Estado siguiente: `010` →

sec-4: Estado: `110` → Salida: `1011` → Estado siguiente: `111` →

sec-5: Estado: `110` → Salida: `1011` → Estado siguiente: `111` →

sec-6: Estado: `111` → Salida: `0001` → Estado siguiente: `000` →

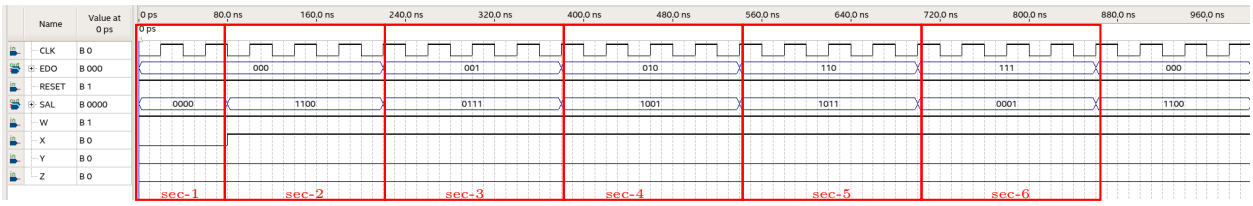


Figure 6: Simulación 1 de la máquina de estados en ModelSim, segmentada por transiciones de estado.

Con esta simulación validamos que la maquina de estados funciona correctamente, con las entradas señaladas y se obtuvierón las salidas esperados en cada transición de estado.

Simulación 2

En esta simulación se consideraron las entradas `X=1, Y=0, Z = 1 y W = 1`.

sec-1: Estado: `000` → Salida: `0000` → Estado siguiente: `000` →

sec-2: Estado: `000` → Salida: `1100` → Estado siguiente: `001` →

sec-3: Estado: `001` → Salida: `0111` → Estado siguiente: `010` →

sec-4: Estado: `010` → Salida: `1001` → Estado siguiente: `110` →

sec-5: Estado: `110` → Salida: `0011` → Estado siguiente: `101` →

sec-6: Estado: `101` → Salida: `0110` → Estado siguiente: `001` →

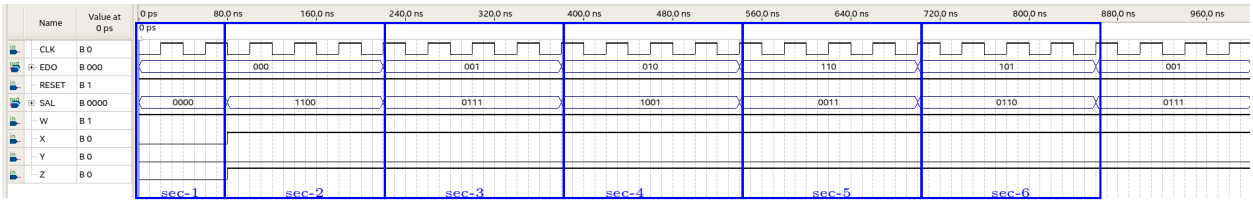


Figure 7: Simulación 2 de la máquina de estados en ModelSim, segmentada por transiciones de estado.

En este camino de simulación, se validó la segunda ruta que nuestra máquina de estados puede tomar. En este no se retorna al estado inicial, si no que regresa al estado `001` y se repite el ciclo de estados `001` → `010` → `110` → `101` → `001`.

Simulacion 3

En esta simulación se consideraron las entradas `X=1, Y=1, Z = 1 y W = 1`. Notar que en este caso el valor de entrada de Z, en realidad no afecta el compirtamiento de este recorrido de la maquina de estados, por lo que se puede considerar que la entrada Z es irrelevante para este camino de simulación.

sec-1: Estado: `000` → Salida: `0000` → Estado siguiente: `000` →

sec-2: Estado: `000` → Salida: `1100` → Estado siguiente: `001` →

sec-3: Estado: `001` → Salida: `0111` → Estado siguiente: `010` →

sec-4: Estado: `010` → Salida: `0110` → Estado siguiente: `011` →

sec-5: Estado: `011` → Salida: `0011` → Estado siguiente: `100` →

sec-6: Estado: `100` → Salida: `1100` → Estado siguiente: `101` →

sec-7: Estado: `101` → Salida: `0110` → Estado siguiente: `001` →

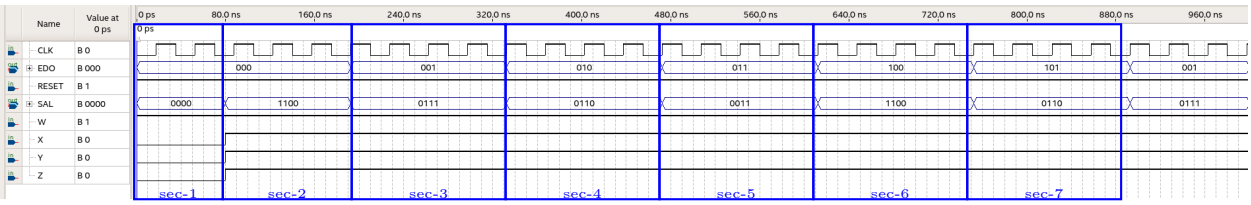


Figure 8: Simulación 2 de la máquina de estados en ModelSim, segmentada por transiciones de estado.

En esta simulación se validó la tercera ruta que nuestra máquina de estados puede tomar. Concluyendo que todos los caminos de la máquina de estados funcionan correctamente y se obtuvieron las salidas esperadas en cada transición de estado.

Conclusiones

Andrea Saldaña Navarrete - 320206384

Durante el desarrollo y simulación de la máquina de estados, se comprobó que el éxito del direccionamiento implícito depende de una transcripción rigurosa de la Tabla de Transiciones a la memoria ROM. Se identificó que un error tipográfico en el orden de concatenación de los bits de salida (colocando las salidas de incremento antes que las de carga) puede alterar por completo el comportamiento del sistema, provocando que la máquina salte de estado en lugar de mantenerse en un ciclo continuo. Esto demostró que la arquitectura de hardware

diseñada funciona correctamente y procesa las señales de los multiplexores de manera exacta, pero exige que la programación de la memoria respete estrictamente el formato de palabra definido (Prueba & VF & Liga & Salidas_Carga & Salidas_Inc) para reflejar fielmente la carta ASM.

Hansel Yael Tepal Briseño - 320078282

El diseño de la máquina mediante direccionamiento implícito presentó un desafío arquitectónico interesante al requerir transiciones incondicionales (como el paso del estado 101 al 001) dentro de un sistema diseñado para siempre evaluar una entrada externa a través de un multiplexor. La solución implementada consistió en utilizar una variable de entrada constante configurada siempre en alto (W=1). Al instruir a la memoria para que el multiplexor evaluara esta señal constante, se forzó al sistema a tomar de manera predecible la ruta de la liga falsa, logrando un salto incondicional sin necesidad de modificar el diseño de los bloques de hardware. Asimismo, se demostró que este comportamiento incondicional también puede lograrse igualando las salidas de carga e incremento y apuntando la liga al estado inmediatamente siguiente, confirmando la flexibilidad del método.

Luis Antonio Ugartechea González - 320223822

En la construcción de la máquina de estados, el método de direccionamiento implícito demostró su eficiencia al reducir los requerimientos de almacenamiento utilizando un único campo de liga en la memoria. La implementación exitosa en hardware requirió un diseño altamente modular, donde una sola palabra de control contiene toda la instrucción de transición y debe ser desempaquetada por un módulo divisor de datos. Esta separación de la información comprobó cómo la arquitectura coordina armónicamente la lógica combinacional y los multiplexores: evaluando dinámicamente la entrada seleccionada contra el bit Verdadero-Falso (VF) para decidir instantáneamente si el sistema toma la trayectoria de incremento hacia el siguiente estado secuencial, o la trayectoria de carga para saltar a la dirección del registro de liga.

1 Bibliografía

[1] D. K. Pérez Pérez, "Reporte Construcción de Máquinas de Estados Usando Memorias", Laboratorio de Organización y Arquitectura de Computadoras, Facultad de Ingeniería, Universidad Nacional Autónoma de México, Semestre 2027-1, 2026.

[2] Facultad de Ingeniería, "Capítulo III: Construcción de máquinas de estados usando memorias", Apuntes de Arquitectura de Computadoras, Universidad Nacional Autónoma de México (UNAM). [En línea]. Disponible en: https://biorobotics.fi-p.unam.mx/wp-content/uploads/Courses/arquitectura_de_computadoras/2021-1/lecciones/Leccion_3_arquitectura_de_computadoras_2021_1.pdf

[3] J. Savage, *Diseño de Microprocesadores*, Facultad de Ingeniería, Universidad Nacional Autónoma de México (UNAM). [En línea]. Disponible en: http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf